What a Success Rate Measures: Configuration, Compute and Sampling in a Robot Learning Benchmark

Yandong Huang
Beijing Information Science and Technology University

November 7, 2026

Abstract

A reported success rate is a function of the method, of how it was configured, and of how it was measured, yet only the first is usually written down. Taking one widely used open-source stack (LeRobot v0.6.1) and its reference task (PushT), we use a 2×2 factorial design with 200 paired episodes per cell to decompose the gap between the published success rate (65.4%) and what the library yields out of the box (33.0%). A single configuration field, `crop_shape`, accounts for the dominant share: enabling it alone raises success from 33.0% to 55.0% (+22.0 percentage points, pp; $p < 0.0001$), and with the remaining fields of the published recipe that cell reaches 63.5%. GroupNorm together with random backbone initialization contributes only +2.0 pp on its own ($p = 0.36$). Three separate experiments show that training budget is not the lever: 2×, 5× and 6× more optimization produce no measurable gain. We further measure the evaluation noise floor with eleven disjoint blocks of 50 initial states: the observed block-to-block spread (±6.0 pp) matches the binomial prediction (±6.2 pp; $\chi^2 = 9.4$, df= 10, $p = 0.50$), so uncertainty is fully accounted for by sample size alone. Finally, the same configuration choice is enough to move a method comparison from indistinguishable ($p = 0.51$) to decisive ($p < 0.0001$), which suggests that configurations must be reconciled before any cross-method claim is made. Taken together, the measurements yield a cost-normalized ordering of interventions: the two largest gains we can produce come from changes that cost no additional compute at all, while the only compute-expensive option we test is measurably ineffective.

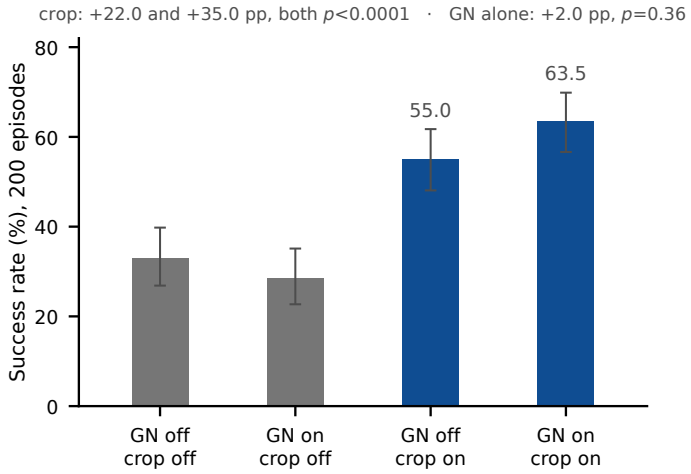


Figure 1: Headline result. Same code, same data, same training budget: the measured success rate differs by thirty percentage points depending only on which configuration fields match the published recipe. Grey, library defaults; blue, `crop_shape` enabled, the rightmost bar additionally restoring GroupNorm and a randomly initialized backbone (the full published recipe). Error bars are Wilson 95% intervals over 200 paired episodes; exact counts and paired tests are in Tables 1 and 2.



Figure 2: Same initial state, two configurations. Top row: the published recipe drives the T-block onto the green target in 118 steps. Bottom row: with the library's shipped defaults the block barely moves and the episode times out at 300 steps. Columns are 0/25/50/75/100% of each episode's recording (evaluation video, episode 1 of the paired 200-episode set); frames are the raw 384×384 renders with no post-processing.

1 Introduction

A practitioner who installs an open-source robot learning library and trains the reference policy on the reference task expects to land near the published number. In our case they do not: the published Diffusion Policy [8] result on PushT is 65.4% [1], whereas the same library with its shipped defaults yields 33.0%, a gap of roughly thirty percentage points, with a training loss that looks entirely healthy. The two numbers come from different checkpoints, 175k and 120k; the budget study in §4.2 shows that doubling ours moves the default cell by 2.0 pp, so the checkpoint difference cannot carry the gap.

The natural first hypotheses are (i) insufficient training, (ii) insufficient data, and (iii) a bug. We test the first directly and reject it; the second is bounded by prior work; and the third turns out to be the wrong frame: no line of code is incorrect. What differs is which of the library's configuration fields are set, and the library neither reports nor warns about the consequences.

This paper makes five contributions:

1. A 2×2 factorial decomposition of a 30-point reproduction gap at 200 paired episodes per cell, attributing it primarily to one configuration field (§4.1).
2. Three independent demonstrations that training budget is not the lever (§4.2).
3. To our knowledge, the first reported evaluation noise floor for this benchmark: eleven disjoint seed blocks, with overdispersion formally tested (§4.3).
4. Evidence that the same configuration choice flips the statistical conclusion of a method comparison (§4.4).
5. A cost-normalized ordering of interventions and a six-step diagnostic protocol for failed reproductions (§5, §6).

We do not claim that any method is better than another; we claim that the measurement is configuration-dependent in a way that is currently unreported.

2 Related Work

Benchmark construction. RLBench [3] and REPLAB [4] standardize tasks and hardware respectively. Neither studies how the *software configuration* of a policy affects measured performance. For imitation learning, [11] varies data quantity, data quality, observation modalities and algorithm family on a fixed task suite; that design varies the training inputs, whereas we vary the library configuration under which a published number is reproduced.

Reproducibility frameworks. MIRRER [5] proposes a conceptual framework for iterated, reproduced and replicated experiments and states that a first step is "simply reporting the resulting performance metrics from several iterated and reproduced experiments and calculating the variance between them." To our knowledge this measurement has not been reported for a robot-learning benchmark. RB2 [6] quantifies a 20% change when the same experiment is run in two different *physical* laboratories and proposes local relative ranking. Our setting has zero physical variation (one machine, one room, identical hardware) and still exhibits a larger shift.

Variance and statistical rigor. In deep reinforcement learning, seed choice, implementation details and the reporting protocol are known to move measured performance by enough to change conclusions [12, 13, 14, 15]. Those studies compare algorithms within one stack; the variation we report arises before training, in configuration fields the library treats as incidental. [15] argue that benchmark variance should be estimated rather than assumed small, which §4.3 does here.

Statistical evaluation. STEP [7] addresses how many episodes are needed to compare two policies in the 10–50 episode regime. Such tests presuppose that the two measurements were taken under comparable conditions. Our results indicate this presupposition can fail even within a single library and a single machine. Observation-contract auditing [16] detects, statically from source, silent loss of state fields across observation pathways. Our mismatch is of a different kind: no field is lost and no code path is defective, so a static contract audit would report both configurations as valid.

Action chunking. The effectiveness of action chunking in imitation learning is acknowledged to lack a complete explanation [17]. We provide a cross-architecture characterization of one of its knobs, the number of actions executed open-loop (§4.5).

3 Experimental Setup

Stack. LeRobot v0.6.1 with four local source patches (see the reproducibility statement). Training and evaluation run on a single consumer GPU (RTX 4060 Laptop, 8 GiB).

Task and data. `lerobot/pusht`: 206 demonstrations, 25650 frames, 96×96 images. Success is defined by the environment as polygon coverage above 0.95; the reported reward is $\text{clip}(\text{coverage}/0.95, 0, 1)$, so reward saturates at 1.0 exactly at the success threshold.

Evaluation protocol. 200 episodes, 300-step horizon, `n_action_steps` = 8 for Diffusion Policy, a fixed evaluation seed, and 22 synchronous environments. Consecutive evaluations with the same seed share the same initial states, so all comparisons reported here are *paired*; we use the exact McNemar test on discordant pairs and Wilson intervals for marginal rates.

Determinism. Two library behaviors had to be neutralized before any comparison was meaningful: cuDNN autotuning (`cudnn.benchmark`) selected different convolution algorithms per process, producing episode-level reward differences of 197.30 versus 63.91 for an identical configuration and seed; and action sampling drew from a global RNG whose state carried over between evaluation batches. Both are patched. With the patches, two independent evaluations of the same checkpoint are bit-identical.

Design. Four training runs share dataset, batch size (64), step count (120k), horizon, and training seed; they differ only in whether `crop_shape` is $[84, 84]$ and whether GroupNorm is enabled together with

Table 1: Success rate (%), 200 episodes per cell, paired initial states. Wilson 95% intervals in brackets.

	crop off	crop on
GroupNorm off (ImageNet backbone)	33.0 [26.9, 39.8]	55.0 [48.1, 61.7]
GroupNorm on (random backbone)	28.5 [22.7, 35.1]	**63.5** [56.6, 69.9]

Table 2: Paired exact McNemar tests over 200 shared initial states.

Contrast	only first succeeds	only second succeeds	p
crop effect, GroupNorm off	27	71	< 0.0001
crop effect, GroupNorm on	20	90	< 0.0001
GroupNorm effect, crop off	43	34	0.362
GroupNorm effect, crop on	34	51	0.082
default → full recipe	20	81	< 0.0001

randomly initialized backbone weights (the two are bound in code and cannot be separated).

4 Results

4.1 A single field carries the gap

Table 1 reports the 2×2. Main effects are +2.0 pp for GroupNorm with a random backbone and +28.5 pp for `crop_shape`; the interaction is +13.0 pp with a bootstrap 95% interval of $[+0.0, +25.5]$, i.e. marginal. Paired McNemar tests (Table 2) confirm that both crop contrasts are decisive while neither GroupNorm contrast is.

A second training seed for the crop-on, GroupNorm-off cell (seed 200000 instead of 100000, all else identical) reaches 116/200 = 58.0% against 110/200 = 55.0% for the first, a difference of +3.0 pp (paired bootstrap 95% interval $[-6.5, +12.5]$). On the shared 200 initial states the two checkpoints agree on only 104 episodes: 96 change outcome, 45 in one direction and 51 in the other (exact McNemar $p = 0.61$). The crop effect therefore survives a change of training seed while the per-episode outcome does not, so what replicates across seeds is the marginal rate rather than the individual trial.

Two consequences follow. First, the configuration field that separates a 33.0% measurement from the published 65.4% is one whose effect is invisible in the training loss. Second, no *single* field closes the gap on its own: `crop_shape` is necessary and accounts for most of it, and only the full published recipe reaches 63.5%, statistically indistinguishable from the published figure measured over 500 episodes.

An upstream fix that misses this field. The library's default for the image encoder and for GroupNorm was changed in a merged pull request in May 2026, whose motivation was that the defaults were "designed for the easier PushT task" [18]. `crop_shape` was not among the changed fields, yet it is the dominant factor here.

4.2 Training budget is not the lever

Table 3 compares two training budgets at matched evaluation. The crop-only cell produces *exactly* the same number of successes at both budgets (110/200). Two further observations point the same way: the full-recipe cell is non-monotonic across five checkpoints (42.0, 48.0, 62.0, 48.0, 64.0%) and flat within the ±13 pp interval of a 50-episode evaluation; and an ACT policy trained for 6× longer (0.8M → 4.8M samples) moves

Table 3: Success rate (%) at two training budgets, 200 episodes each.

Cell	60k steps	120k steps
default	31.0 (62/200)	33.0 (66/200)
GroupNorm only	26.5 (53/200)	28.5 (57/200)
crop only	55.0 (110/200)	55.0 (110/200)

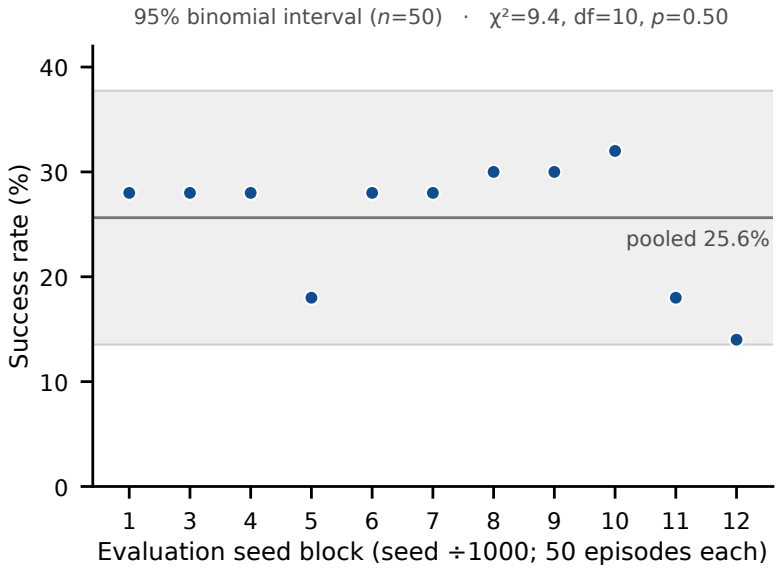


Figure 3: Evaluation noise floor. Eleven disjoint blocks of 50 initial states, same checkpoint and configuration. Bars are per-block success rates; the shaded band is the 95% binomial interval implied by $n = 50$ at the pooled rate (25.6%). All eleven blocks fall inside the band, so the spread is accounted for by sample size alone ($\chi^2 = 9.4$, df= 10, $p = 0.50$).

from 27.0% to 29.5% (31 vs 36 discordant, $p = 0.63$). Two model families, three budget multipliers, no measurable gain.

4.3 The evaluation noise floor

Eleven disjoint blocks of 50 initial states, same checkpoint and identical configuration, give success rates of 28, 32, 18, 14, 28, 28, 18, 28, 28, 30 and 30%. The block-to-block standard deviation is ±6.0 pp against a binomial prediction of ±6.2 pp (ratio 0.97); a χ^2 overdispersion test gives $\chi^2 = 9.4$ with df= 10 ($p = 0.50$). There is no evidence of extra-binomial variance: at $n = 50$, uncertainty is entirely attributable to sample size. This is, to our knowledge, the measurement called for by [5] and by the variance-reporting framework of [15], but not previously reported, and it justifies the use of binomial / Wilson intervals throughout. This statement concerns evaluation noise at a fixed checkpoint. Training adds a second, larger source of variation, measured in §4.1: two seeds of the same cell differ by 3.0 pp in the marginal rate yet disagree on 96 of 200 individual episodes. The binomial interval says how precisely a fixed policy is measured, not how reproducible a training run is.

Table 4: Interventions measured in this study, ordered by cost. "n.s." marks a difference that did not reach significance. All figures are from the same task and hardware; they are a ranking within this study, not a universal law.

Intervention	Δ success	Extra GPU-h	Nature of change
`n_action_steps` 50→10 (SmolVLA)	+17 pp	0	inference-time
`n_action_steps` 1→10 (ACT)	+27 pp	0	inference-time
`crop_shape` none→ [84, 84]	+22 pp	0	configuration
episode budget 300→600	+32 pp	0	task definition
training ×2 (DP, 60k→120k)	+2.0 pp (n.s.)	+7.3	optimization
training ×6 (ACT, 0.8M→4.8M)	+2.5 pp (n.s.)	+10	optimization
dataset reduced to 1/4	−24 pp	—	data

4.4 What this does to a method comparison

Holding the evaluation protocol fixed, we compare ACT [9] (100k and 200k training steps) against Diffusion Policy [8] under two configurations. With the library defaults, DP at 33.0% and ACT at 29.5% are indistinguishable (37 vs 44 discordant, $p = 0.51$). With the published recipe, DP at 63.5% dominates (13 vs 81, $p < 0.0001$). The configuration choice alone moves the comparison from a null result to a decisive one.

We emphasize what this does *not* establish. The ACT policy was not configuration-tuned: ACT has no published PushT recipe to restore, so we cannot exclude that a better-configured ACT would perform differently. The claim is therefore not about the relative merit of the two methods, but about the fragility of the comparison itself.

4.5 Receding-horizon length across architectures

For ACT, the number of actions executed before replanning has a sharp interior optimum: success is 0.5, 1.5, 3.0, 10.5, 17.0, **27.0**, 18.5, 6.0 and 1.0% for `n_action_steps` of 1, 2, 4, 6, 8, 10, 20, 50 and 100 respectively. For SmolVLA [10], a 450M language-conditioned policy fine-tuned on the same data (the only model here that takes text as input; ACT and Diffusion Policy are purely visuomotor), the ordering is 6.5% at `n`=50, 8.0% at 20, and 23.5% at 10 and 21.0% at 5; the last two are indistinguishable (33 vs 28 discordant, $p = 0.61$), while ten against fifty is decisive (40 vs 6, $p < 0.0001$). Both architectures peak between five and ten actions executed open-loop and fall by a factor of three to four by fifty, even though the prediction horizons (100 and 50) and model sizes (80M and 450M) differ by a factor of five, which suggests the quantity is set by the task's temporal scale rather than by the architecture. For ACT the peak is resolved: ten beats eight (41 vs 21, $p = 0.015$) and twenty is borderline (43 vs 26, $p = 0.053$). All settings use the same 200-episode protocol as the rest of the paper.

5 Where to Spend: A Cost-Normalized View

Because every quantity in this paper was measured under a fixed compute budget, we can order the interventions by what they buy per unit of compute (Table 4). The ordering is stark.

The two largest gains that survive scrutiny are obtained by changing a value in a configuration file or an inference-time parameter. Neither costs a single additional GPU-hour. The one intervention that does cost compute, longer optimization, produces differences that our sample sizes cannot distinguish from zero, at two different multipliers and in two different model families.

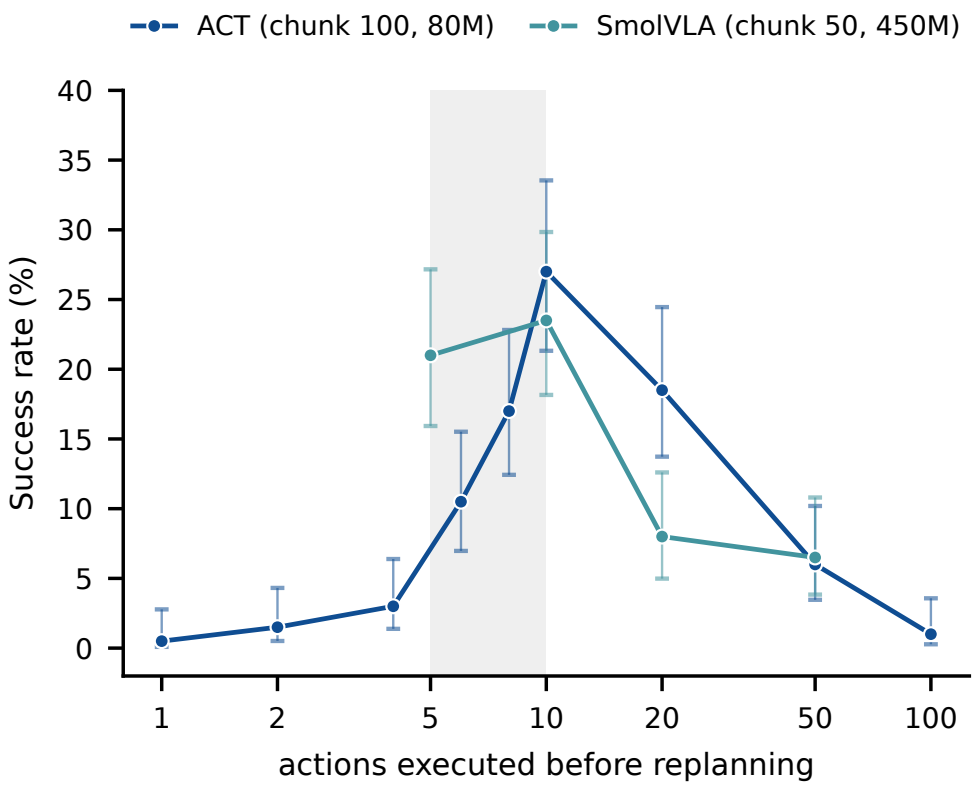


Figure 4: A shared interior optimum for open-loop execution. Success rate against the number of actions executed before replanning, for ACT (chunk 100, 80M parameters) and SmolVLA (chunk 50, 450M parameters); 200 episodes per point, Wilson 95% intervals. Both architectures peak at 5–10 actions and fall by a factor of three to four by 50, despite a fivefold difference in prediction horizon and model size.

We deliberately exclude the 300→600 episode result from the "gain" column even though it is the largest number in the table: raising the horizon changes the task definition, so the resulting success rate is not comparable to the published 300-step protocol. It is nevertheless informative: a third of the failures at the standard horizon are cases the policy could have solved given more steps, which is itself an argument that the reported number is protocol-dependent.

6 A Diagnostic Protocol for Failed Reproductions

The steps below are the ones that actually located our gap, ordered by the ratio of information gained to time spent. Each addresses a specific symptom.

1. **Establish determinism before trusting any comparison.** Evaluate the same checkpoint twice with identical settings. If the numbers differ, the comparison machinery is broken and no configuration study is meaningful. In our stack this required disabling cuDNN autotuning and re-seeding the action-sampling RNG once per evaluation batch.

2. **Verify the metric, not just the number.** Confirm whether the reported quantity is a thresholded success indicator or a clipped continuous score; the two are not monotonically recoverable from one another, and a saturating score can rank methods differently from the success rate.

3. **Diff configurations against the reference artifact.** Field-by-field comparison of the stored training configuration against the published checkpoint's configuration located all four fields in which the two differ, one of them inert with a single camera. Visual inspection of two YAML files does not scale; a sorted diff does.

4. **Use a factorial rather than one-factor-at-a-time.** A single-factor sweep would have concluded that the normalization change was useless (it costs 4.5 pp in the crop-off cell and gains 8.5 pp in the crop-on cell, averaging +2.0 pp) and stopped. The 2×2 revealed that the effect of the input crop depends on the normalization choice.

5. **Pair the evaluations and report n with an interval.** Reusing the same initial states across configurations multiplies the sensitivity of the comparison; reporting a bare success rate with $n = 50$ hides a ± 12 pp interval that is larger than most claimed improvements.

6. **Check every input the policy conditions on, text included.** A language-conditioned policy evaluated without the instruction it was trained with runs silently: the command succeeds, metrics are written, nothing warns. In our stack the missing variable defaulted to the empty string, so a policy conditioned on it lost a factor of three to four in success rate with no traceback. Assert that the instruction, the camera key names and the state fields are exactly the ones the checkpoint was trained with.

7 Discussion

The distinction that matters is between two kinds of mismatch. An *implementation defect* loses or reorders information: a field is dropped, a key is resorted, a code path is skipped. Such defects are in principle detectable without running anything, and a static contract audit [16] is the right tool for them. A *convention mismatch* is different in kind: both configurations are legal, both train to a healthy loss, neither loses information, and a static audit would pass both. What fails is the comparison against a reference produced under the other convention.

This is why the gap we measure cannot be found by inspection. It can only be found by knowing which reference one intends to reproduce, recovering that reference's configuration, and measuring the difference under a paired protocol. The first two are bookkeeping; the third is statistical. The protocol in §6 is an attempt to make all three routine, and the cost table in §5 is an argument for doing them before spending compute.

8 Limitations

Single task, simulation only. All experiments use PushT. Generalization to other environments is untested and is the most important open question.

Two training seeds for one cell only. The crop-on, GroupNorm-off cell was trained twice (55.0% and 58.0%, $p = 0.61$); the other three cells rest on a single seed each, so the seed-to-seed band is estimated from one cell rather than characterized across the factorial.

ACT was not configuration-tuned, as discussed in §4.4.

The reproduction target is a single published number for a single checkpoint, and its own value (65.4%) was measured with 500 episodes and a 175k-step checkpoint.

9 Reproducibility Statement

All training and evaluation scripts, the raw per-episode results for 65 evaluation runs, the exported source patches, and the result tables are available at `https://github.com/JacksonHYD/vla-pusht-repro-artifact`, archived at doi:10.5281/zenodo.24034428. Model weights are archived at `https://huggingface.co/JacksonHYD/pusht-repro-checkpoints`. The four source patches (deterministic convolution, per-batch RNG seeding, state-less ACT, per-episode step recording) are provided as a `.patch` file against LeRobot commit `7e241bd6`; without them the numbers below are not reproducible.

References

[1] LeRobot. *diffusion_pusht* model card, huggingface.co/lerobot/diffusion_pusht. Reports 65.4% success over 500 episodes at the 175k-step checkpoint.

[2] LeRobot: An Open-Source Library for End-to-End Robot Learning. arXiv, 2026.

[3] S. James, Z. Ma, D. R. Arrojo, A. J. Davison. RLBench: The Robot Learning Benchmark & Learning Environment. arXiv:1909.12271, 2019.

[4] B. Yang, J. Zhang, V. Pong, S. Levine, D. Jayaraman. REPLAB: A Reproducible Low-Cost Arm Benchmark Platform for Robotic Learning. ICRA 2019. arXiv:1905.07447.

[5] A. Norton, B. Flynn. Towards Using Multiple Iterated, Reproduced, and Replicated Experiments with Robots (MIRRER) for Evaluation and Benchmarking. arXiv:2408.04736, 2024.

[6] S. Dasari et al. RB2: Robotic Manipulation Benchmarking with a Twist. arXiv:2203.08098, 2022.

[7] D. Snyder et al. Is Your Imitation Learning Policy Better than Mine? Policy Comparison with Near-Optimal Stopping. arXiv:2503.10966, 2025.

[8] C. Chi, Z. Xu, S. Feng, et al. *Diffusion Policy: Visuomotor Policy Learning via Action Diffusion*. RSS 2023. arXiv:2303.04137.

[9] T. Z. Zhao, V. Kumar, S. Levine, C. Finn. *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware*. RSS 2023. arXiv:2304.13705.

[10] M. Shukor, D. Aubakirova, F. Capuano, et al. *SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics*. arXiv:2506.01844, 2025.

[11] A. Mandlekar, D. Xu, J. Wong, et al. *What Matters in Learning from Offline Human Demonstrations for Robot Manipulation*. CoRL 2021. arXiv:2108.03298.

[12] P. Henderson, R. Islam, P. Bachman, et al. *Deep Reinforcement Learning that Matters*. AAAI 2018. arXiv:1709.06560.

[13] L. Engstrom, A. Ilyas, S. Santurkar, et al. *Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO*. ICLR 2020. arXiv:2005.12729.

[14] R. Agarwal, M. Schwarzer, P. S. Castro, et al. *Deep Reinforcement Learning at the Edge of the Statistical Precipice*. NeurIPS 2021. arXiv:2108.13264.

[15] X. Bouthillier, P. Delaunay, M. Bronzi, et al. *Accounting for Variance in Machine Learning Benchmarks*. MLSys 2021. arXiv:2103.03098.

[16] Observation-contract auditing for valid modality comparison in robot manipulation benchmarks. *Scientific Reports*, 2026. doi:10.1038/s41598-026-76821-1.

[17] Q. Li, Z. Zhou, S. Levine. Reinforcement Learning with Action Chunking. NeurIPS 2025. arXiv:2507.07969.

[18] huggingface/lerobot pull request #3202, "feat(policy): use pretrained vision encoder weights by default for diffusion and vqbet", merged 2026-05-07.